

Relazione Progetto C3 — Simulatore Multi-Server

Descrizione del progetto

Il progetto C3 estende il simulatore ad evento discreto fornito come esempio trasformandolo da un sistema con singolo server (coda G/G/1) a un sistema con N server paralleli, ciascuno con la propria coda FIFO indipendente, arrivi Poissoniani e tempi di servizio esponenziali.

Sono supportate tre politiche di routing dei pacchetti in ingresso:

- Random — il server viene scelto uniformemente a caso
- Round-Robin — i server vengono serviti in rotazione ciclica
- Shortest Queue — il pacchetto viene inviato al server con il minor numero di pacchetti

File modificati rispetto all'esempio

Nel file `global.h` originale veniva definito un singolo seme SEED per la generazione dei numeri casuali. Nella versione C3 sono stati introdotti tre semi distinti (`SEED_ARR` per gli arrivi, `SEV_SRV` per i tempi di servizio e `SEED_RT` per le decisioni di routing) per garantire la mutua indipendenza statistica delle estrazioni. Inoltre, sono state definite le costanti per identificare le tre politiche di routing e la costante `MAX_N` impostata a 64 per porre un limite superiore al numero di server inserito dall'utente.

Nei file `buffer.h` e `buffer.c` la classe che rappresenta il buffer di un server è stata estesa. Rispetto alla versione originale dell'esempio che tracciava solo i contatori del delay e dei pacchetti completati, in C3 sono stati aggiunti i campi `size` per tracciare la dimensione corrente della coda d'attesa, `in_service` per puntare al pacchetto in corso di servizio, ed `area` e `last_update` per calcolare l'integrale temporale dell'occupazione del server. Sono stati inoltre creati il metodo `update_area` per aggiornare tale integrale temporale, il metodo `reset_counters` per azzerare le statistiche ad inizio run e la funzione `station` per ottenere l'occupazione complessiva.

Nei file `event.h` ed `event.c` le classi che gestiscono gli eventi sono state ristrutturare per gestire il multiserver. La classe `arrival` non fa più riferimento a un buffer specifico ma determina la destinazione a runtime invocando la funzione `pick_server`, la quale implementa gli instradamenti random, round-robin e shortest queue. Per l'instradamento shortest queue, in caso di parità viene utilizzato il campionamento a serbatoio per ripartire uniformemente il carico senza favorire i server con indice più basso. La classe `service` porta ora l'indice del server che ha completato il pacchetto in modo da campionare il rispettivo tempo di servizio `mu_inv` ed aggiornare le statistiche specifiche.

I file `queue.h` e `queue.c` dell'esempio, che simulavano una coda a server singolo, sono stati sostituiti da `mqueue.h` e `mqueue.c`. Questa nuova classe alloca dinamicamente un array di N buffer ed N moduli statistici per tracciare il tempo medio di sojourn dei pacchetti, l'occupazione L_i di ciascun server e l'indice di sbilanciamento del carico tra i vari server. Anche la fase di input dei parametri è stata riscritta per accogliere λ , i valori di μ per ogni server e la scelta della politica di routing.

Nel file `main.c` la modifica è minima e consiste nell'istanziare la nuova classe `mqueue` al posto

della classe queue originale, aggiornando anche le informazioni mostrate a schermo all'avvio del programma.

I file `calendar.h`, `calendar.c`, `simulator.h`, `simulator.c`, `packet.h`, `packet.c`, `stat.h`, `stat.c`, `rand.h`, `rand.c`, `easyio.h` ed `easyio.c` sono stati ereditati direttamente dal progetto senza subire alcuna modifica.

Come compilare ed eseguire

```
cd reti/C3
make
./c3sim -o risultati.txt -t trace.txt
```

Il simulatore chiede interattivamente:

1. Modello di arrivo (solo Poisson)
2. Numero di server N
3. Tasso di arrivo λ (pkt/s)
4. Modello di servizio (solo esponenziale)
5. Tasso di servizio μ_i per ogni server i
6. Politica di routing (1=random, 2=round-robin, 3=shortest-queue)
7. Durata transitorio (s)
8. Durata di ogni run (s)
9. Numero di run

Output prodotto

I risultati includono, con intervallo di confidenza al 95%:

- Sojourn time medio W (tempo medio di sistema per pacchetto)
- Occupazione media L_i per ciascun server (coda + in servizio)
- Indice di sbilanciamento = $\max_i(L_i) - \min_i(L_i)$